Large Language Models I

MY459 Computational Text Analysis and Large Language Models

https://lse-my459.github.io/

Friedrich Geiecke

Today

- Introduction to large language models (LLMs)
- At a fundamental level, current LLMs such as ChatGPT are simply autoregressive functions which iteratively predict the next word/token, starting with a sequence of input words/tokens
$$Pr(x_{T+1}|x_T, \ldots, x_1)$$
- The aim of the last two weeks of the course is to develop an understanding of these functions and their applications

Outline

- Foundations
- Sequence models and recurrent neural networks
- The transformer architecture
- Large language (base) models

From text to numbers

- Before we begin with transformers and language models, it is helpful to briefly review some foundations
- How do we move from texts to numerical representations that are processed in neural networks?

Stylised example: A dataset of four patents

- Document 1: "invention"
- Document 2: "a good invention"
- Document 3: "a better invention"
- Document 4: "a less good invention"

Tokenisation

- For simplicity, assume we tokenise at the word level, then there are five tokens in total: "a", "invention", "better", "good", "less"
- A tokenisation dictionary allows to map strings to integers and back

```
{
    "a": 0,
    "invention": 1,
    "better": 2,
    "good": 3,
    "less": 4
}
```

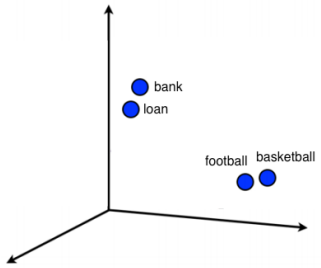
- Assuming this tokeniser, the document "a good invention" becomes `[0, 3, 1]`

Modern tokenisers

- Internally, computers process characters as sequences of bits (0/1); 8 bits are one byte
- Idea of byte-pair encoding (BPE): Start with the 256 possible bytes as the initial token vocabulary, then iteratively merge the most frequent consecutive token pair into a new token until a target vocabulary size is reached
- The recent tokeniser of language models such as *GPT-4o* is *o200k_base* from the library *tiktoken*, it has ca. 200,000 unique tokens
- For example, "london school of economics" becomes $[75, 8552, 3474, 328, 48229]$
- `https://tiktokenizer.vercel.app/?model=o200k_base`

Embeddings

- Tokens are encoded with embeddings that represent their meaning as lists of numbers



- word2vec (Mikolov et al., 2013) or GloVe (Pennington et al., 2014) encoded words as static vectors (last week)

- Modern neural networks learn token embeddings as a byproduct of training, and their hidden states become context-dependent embeddings (this week)

Outline

- Foundations
- Sequence models and recurrent neural networks
- The transformer architecture
- Large language (base) models

Neural language models

- Research on autoregressive neural language models is not new: Bengio et al. (2003) predicted the next word given a short sequence of previous words $Pr(x_{T+1}|x_T, \ldots, x_1)$

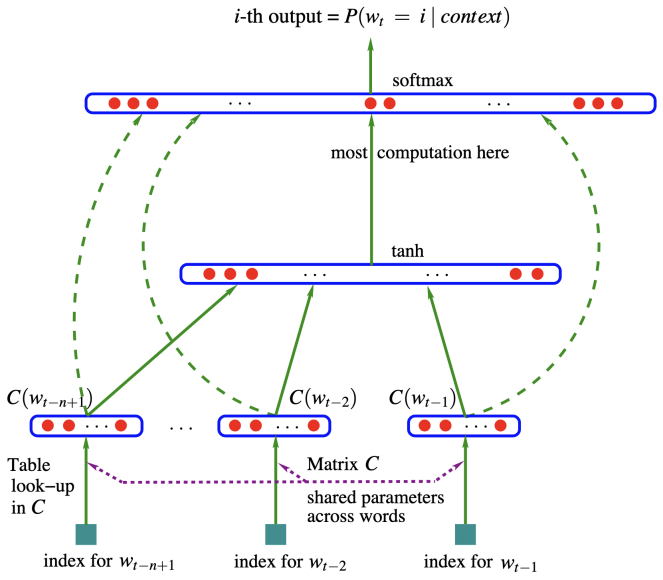


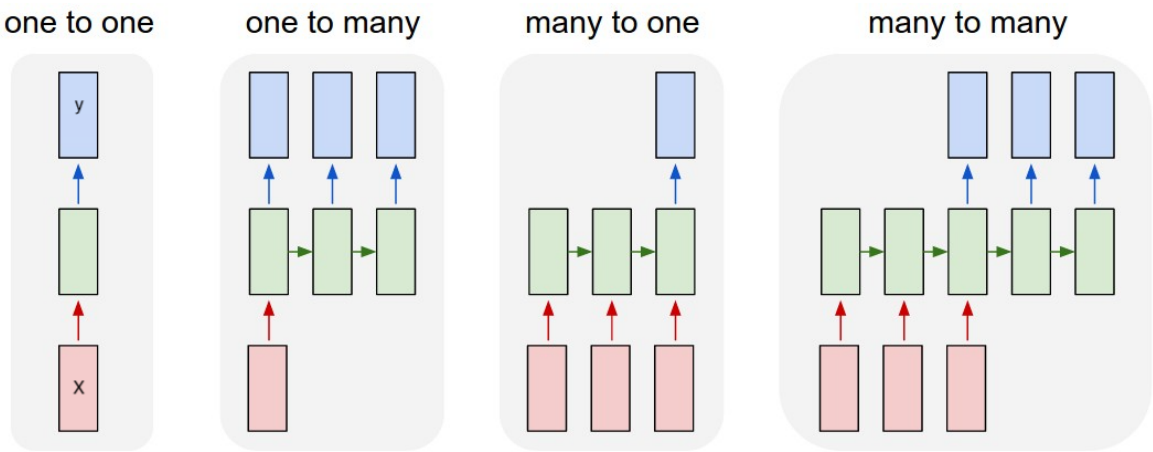
Figure 1 from Bengio et al. (2003)

- This task was originally attempted with simple feedforward neural networks (MLPs) or recurrent neural networks (RNNs)

Recurrent neural networks

- RNNs naturally allow processing sequences of inputs and predict individual items or sequences of outputs
- They have been used in machine translation, sentence completion, sentiment analysis, image captioning, etc.

Common variants of recurrent neural networks



Source: From Andrej Karpathy's blog; slightly edited

- Arrows are functions/transformations, rectangles are vectors, green rectangles hold states
- One to one: Standard feedforward neural network
- One to many: RNN that e.g. takes an image as input and then outputs a sentence describing it
- Many to one: RNN that e.g. inputs a sequence of words and outputs a sentiment label
- Many to many: RNN that e.g. inputs a sentence in one language and outputs it in another language

Challenges

- Baseline architectures struggle to "remember" long-term dependencies in sequences
- Common types of RNNs such as Long Short-Term Memory (LSTM) or those with Gated Recurrent Units (GRUs) are based on cells which improve the model's ability to recall information
- Challenges remain: Vanishing/exploding gradients, encoder-decoder bottlenecks, limited parallelisability of sequential computation, ...
- Today, transformers have taken over a lot of these tasks and will be the focus of this lecture
- If you are interested in the topic of RNNs, very good discussions are `https://karpathy.github.io/2015/05/21/rnn-effectiveness/` or `https://colah.github.io/posts/2015-08-Understanding-LSTMs/`

Introducing the attention mechanism

- A key concept in modern architectures is *attention*
- It was actually introduced to improve RNNs in Bahdanau et al. (2014): *"we conjecture that the use of a fixed-length vector is a bottleneck in improving the performance of this basic encoder-decoder architecture, and propose to extend this by allowing a model to automatically (soft-)search for parts of a source sentence that are relevant to predicting a target word ..."*
- Attention mechanisms were frequently employed in RNNs afterwards
- Eventually, it was shown that attention combined with simpler neural network components was sufficient

Outline

- Foundations
- Sequence models and recurrent neural networks
- The transformer architecture
- Large language (base) models

Attention is all you need (Vaswani et al., 2017)

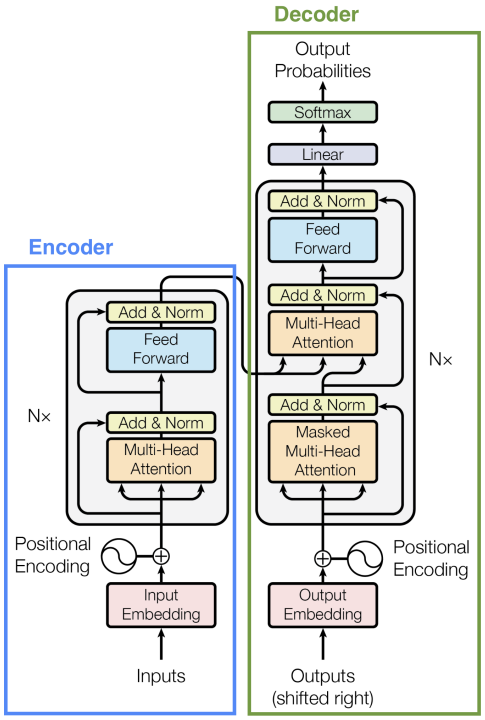


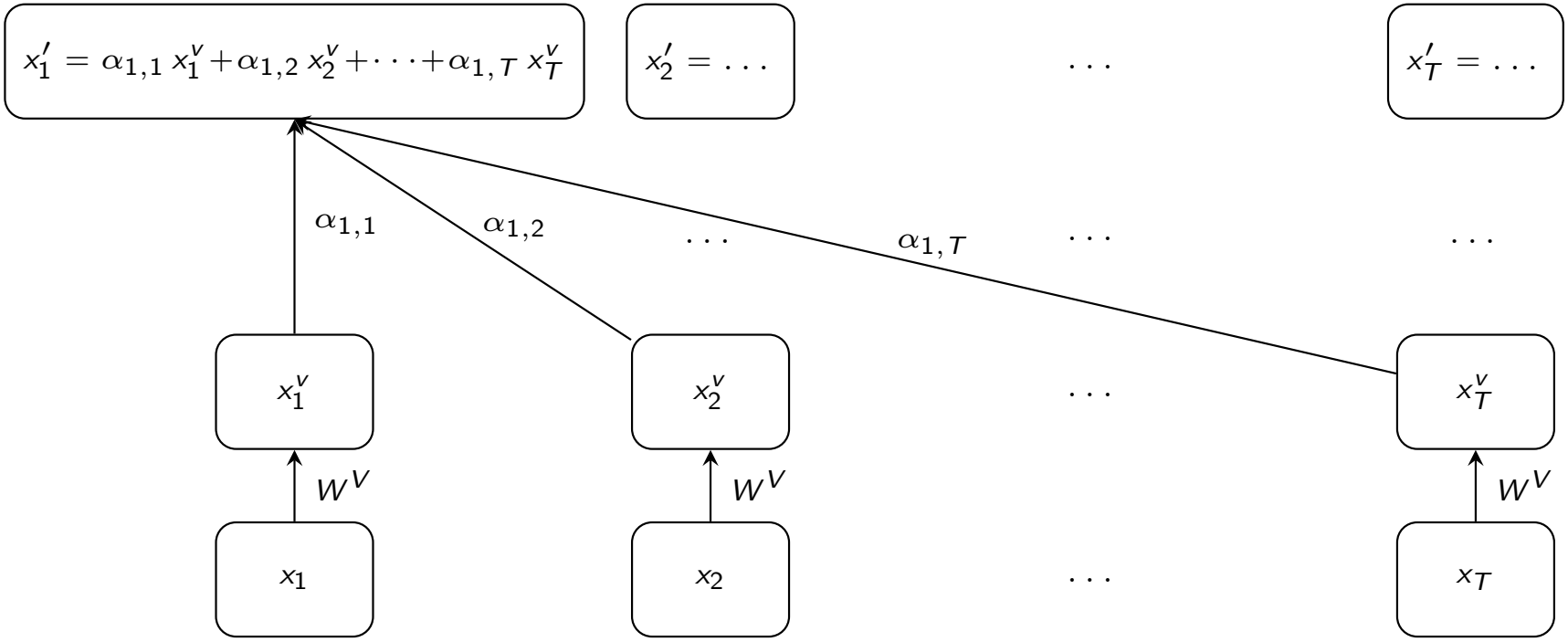
Figure 1 from Vaswani et al. (2017) with added encoder and decoder labels

- Introduced the *transformer* architecture used across machine learning fields today
- The paper was on machine translation, but today encoders (e.g. BERT) and decoders (e.g. GPT, Llama) are often used separately

Fundamental components

- A transformer consists of two fundamental components which are repeated:
  1. Attention
  2. Feedforward neural network
- Before we focus on LLMs, we will discuss these two components more generally
- Imagine you have encoded an input sequence into T static input embeddings, for example, "Language models are interesting" would become x_1, x_2, x_3, x_4
- Attention and feedforward neural network blocks could be applied to this sequence

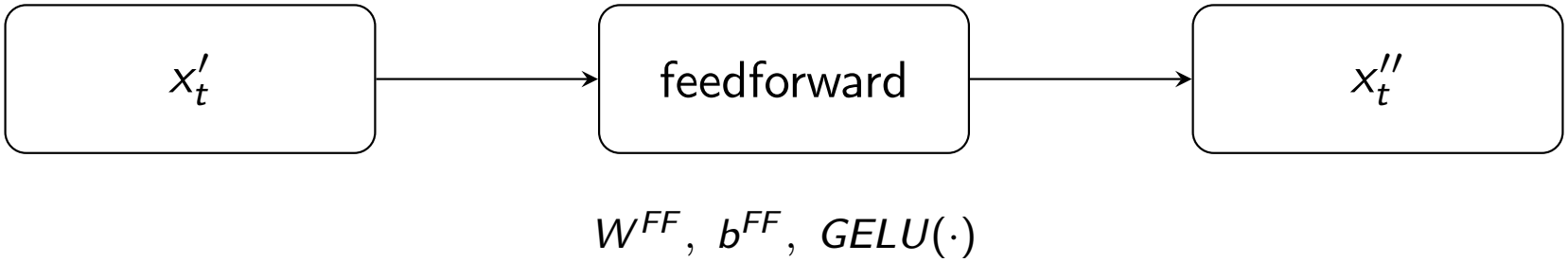
(Self-)attention basics



- Given a value weight matrix W^V, x^v_t is a linear transform: $x^v_t = W^V x_t$
- **Attention simply computes weighted averages over these** x^v
- x'_2 is computed analogously to x'_1, i.e. $x'_2 = \alpha_{2,1}\, x^v_1 + \alpha_{2,2}\, x^v_2 + \cdots + \alpha_{2,T}\, x^v_T$, so is x'_3, etc.
- Weights $\alpha(W^Q, W^K, x)$ are functions of so-called query and key matrices (details) that are learned through gradient descent alongside W^V

Feedforward neural network

- The second fundamental component of the transformer is a simple feedforward neural network



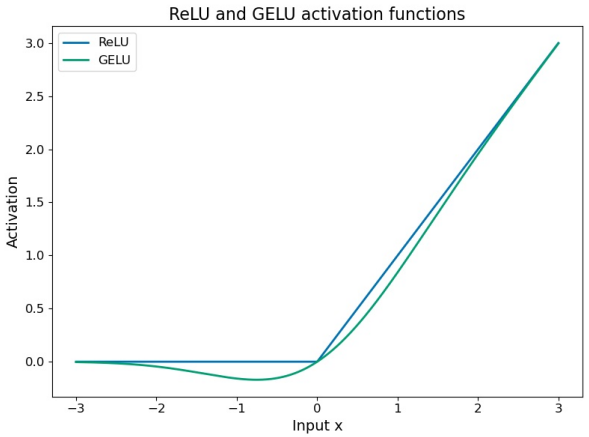
- This is the key non-linear transformation of the architecture
- Unlike in the attention block, each embedding vector x_t' is processed separately
- Combined weight matrices and biases of the network are denoted as W^{FF}, b^{FF} for easier notation

ReLU and GELU

- Transformers often use a GELU activation
- ReLU: $f(x) = \max(0, x)$
- GELU:

$$f(x) = x\,\Phi(x) \approx 0.5\,x\left(1 + \tanh\left[\sqrt{\frac{2}{\pi}}\left(x + 0.044715\,x^3\right)\right]\right),$$

  where $\Phi(x)$ is just the CDF of a standard normal distribution
- Retains small negative values and can lead to improved gradient flow



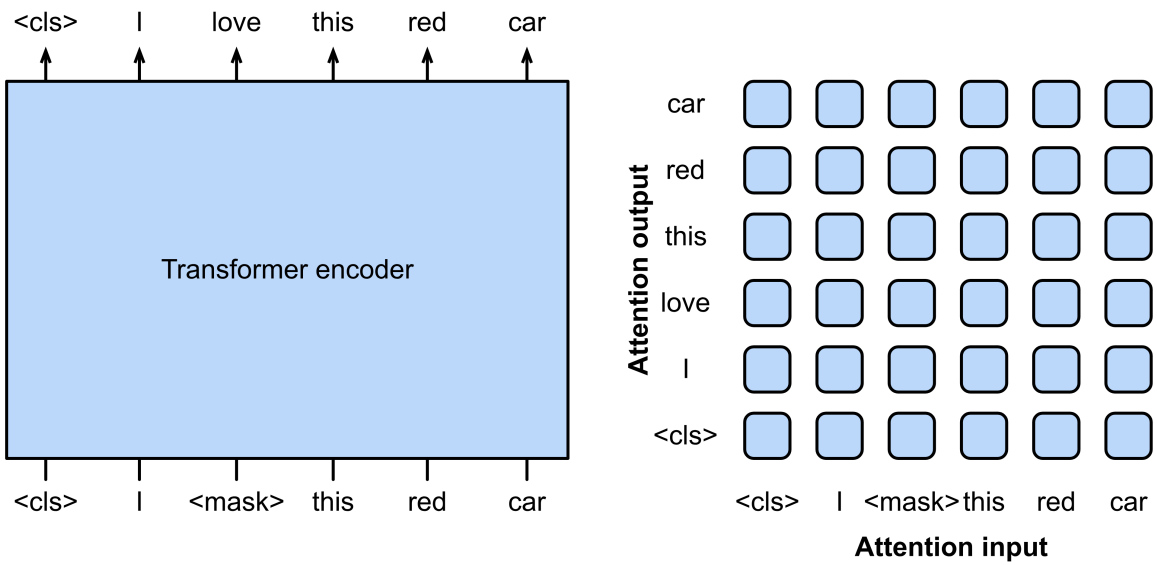
Transformer blocks

- One transformer block combines attention and feedforward network, and adds other features such as multiple attention heads, norms, or skip connections (more details later)
- After applying N transformer blocks, the input embeddings x_1, x_2, x_3, x_4 have become rich contextual representations of the underlying tokens
- The resulting four ouput embeddings can be used in various down-stream tasks

Encoder transformers

- Attention was discussed as fully connected so far - each token attended to each other token
- The embedding for "models" in our exemplary sentence could depend on "Language" as well as "are"
- This is a feature of encoder transformers which yield bidirectional contextual token embeddings and can e.g. be fine-tuned for classification tasks
- For example, BERT (Devlin et al., 2018) or RoBERTa (Liu et al., 2019)
- Parameters are learned by minimising loss functions arising from a task such as masked language modelling or next sentence prediction (BERT)

Encoder-only attention



From `https://en.wikipedia.org/wiki/BERT_(language_model)`

Outline

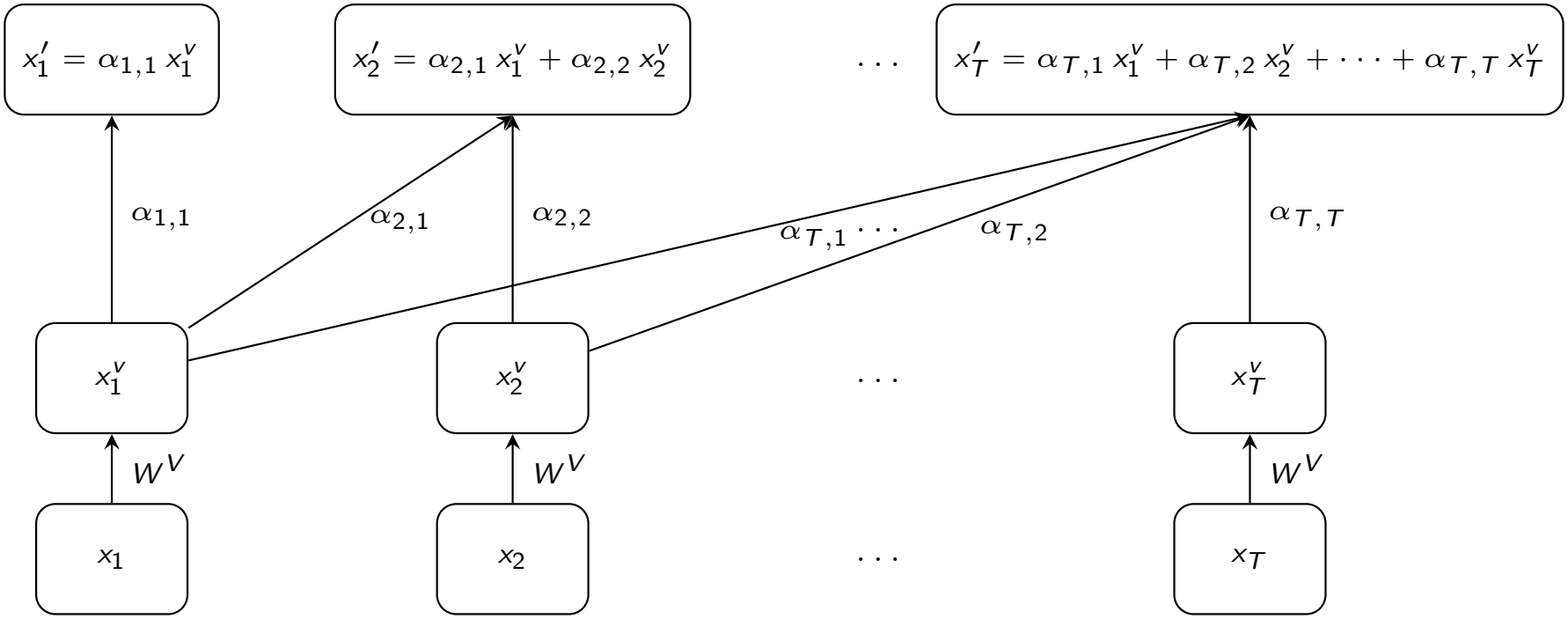
- Foundations
- Sequence models and recurrent neural networks
- The transformer architecture
- Large language (base) models
  - Overview
  - Pre-training
  - Inference

Large language models (LLMs)

- The transformers which we are going to discuss in more detail are autoregressive models of language based only on decoders, i.e. current LLMs
- Their objective remains to predict $Pr(x_{T+1}|x_T, \ldots, x_1)$ similar to Bengio et al. (2003)
- However, today's transformer-based functions are much more flexible with many billions of parameters and attention contexts commonly spanning hundreds of thousands of tokens $x_T, \ldots, x_1$

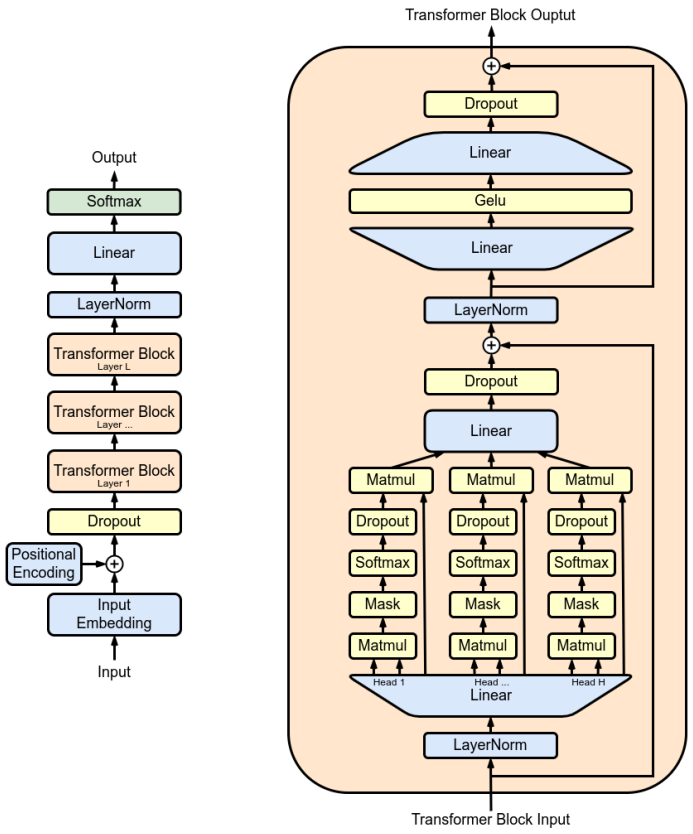
Key difference to encoder transformers: Causal/masked self-attention in decoders

- Attention is only backward-looking as the next token is predicted



~GPT-2 architecture

- We will discuss fundamentals of GPT-2 as an example (Radford et al., 2019)



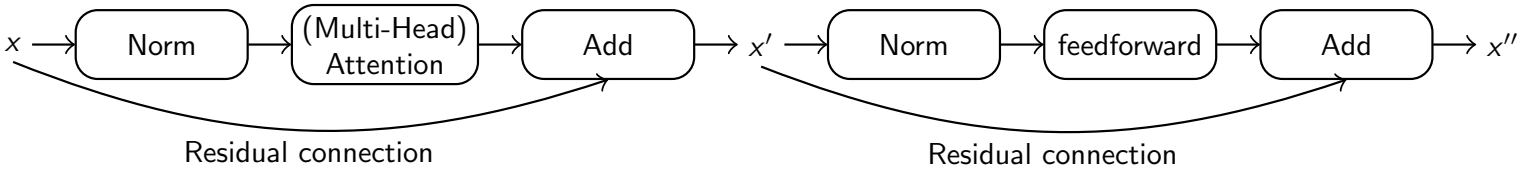
From `https://en.wikipedia.org/wiki/Generative_pre-trained_transformer`

Forward pass (I/III)

- To understand key parts of this architecture, let us go through one forward pass (assuming for now that all weight matrices and biases have only been initialised randomly)
- Imagine the text example from before is now used as a prompt to an LLM: "Language models are interesting"
- With *tiktoken o200k_base* tokeniser, split into an input sequence of $T = 4$ tokens: $[17238, 7015, 553, 9559]$
- Look up initial embeddings $x_{17238}, x_{7015}, x_{553}, x_{9559}$
- Add a positional encoding to each: $x_t = x_{token\ id} + x_t^{pos}$
- The resulting input embeddings enter the transformer blocks (next slide)

Forward pass (II/III)

- Apply $N\times$ transformer blocks



- Details:
  - Norms (typically *layer normalisation*) standardise each embedding vector to have zero mean and unit variance (abstracting from scale and shift parameters here), preventing the activations from growing too large or too small across layers and stabilising gradient flow during training
  - In each block, several attention heads are applied in parallel and the outputs are combined into one vector for each input embedding
  - *Residual/skip connections* add the input of each attention and feed-forward sublayer directly to its output, allowing gradients to flow back through the network, both through the transformations and via a direct path bypassing them, making training significantly more stable

Forward pass (III/III)

- After the last transformer block, another weight matrix W^{vocab} computes $W^{vocab} \cdot x^N = \mathit{logits}$ and maps the embeddings into a vector with $\sim$200,000 dimensions (if we used *o200k_base*)
- A softmax is applied to this logits vector to obtain predicted probabilities of next tokens
- That completes the forward pass. Next, we will study how the parameters are learned

Outline

- Foundations
- Sequence models and recurrent neural networks
- The transformer architecture
- Large language (base) models
  - Overview
  - Pre-training
  - Inference

Training

- Conceptually, predicting the next token is a classification problem
- We simply minimise the cross-entropy loss function discussed in previous weeks: $L(y_i, p_i) = -\sum_{k=1}^{K} y_{i,k} \log(p_{i,k})$
- Gradients of the loss are computed with the chain rule with respect to all parameters of the model:
  - Input embeddings for tokens and positions (unless encoded otherwise): $W^{token\ embed}$ and $W^{position\ embed}$
  - In each transformer block: W_h^V, W_h^Q, W_h^K for all attention heads h (and W^O to combine the outputs of the heads), as well as weights and biases W^{FF}, b^{FF} for the feedforward network
  - A weight matrix W^{vocab} that maps final embeddings into output tokens (unless tied to $W^{token\ embed}$)
  - Parameters for layer normalisations

Pre-training data

- Training of LLMs currently starts with a so-called *pre-training stage*, where the model is trained on very large amounts of curated (internet) text data to predict the next token
- Teams of researchers work only on assembling suitably high-quality internet datasets for pre-training
- One public example is `https://huggingface.co/spaces/HuggingFaceFW/blogpost-fineweb-v1`
- Models are trained on GPU cluster with batches of text data to predict the next token in sequences of different lengths in parallel

Pre-training: Further details

- This first stage of training can be conceptualised as a kind of lossy compression of internet text data into the model's floating point parameter values
- As an example, a Llama 2 model variant with ∼70 billion parameters was trained on ∼10 terabytes of internet text data. Its resulting parameters can be stored in a ∼140GB file
- Training required ∼6,000 GPUs for ∼12 days and may have roughly implied a total cost in the order of $2M at the time (see Karpathy, 2023a, for further discussion)

Scaling laws

- The test loss of the base model turns out strongly correlated with many downstream capabilities
- It tends to decrease in compute, dataset size, and parameters

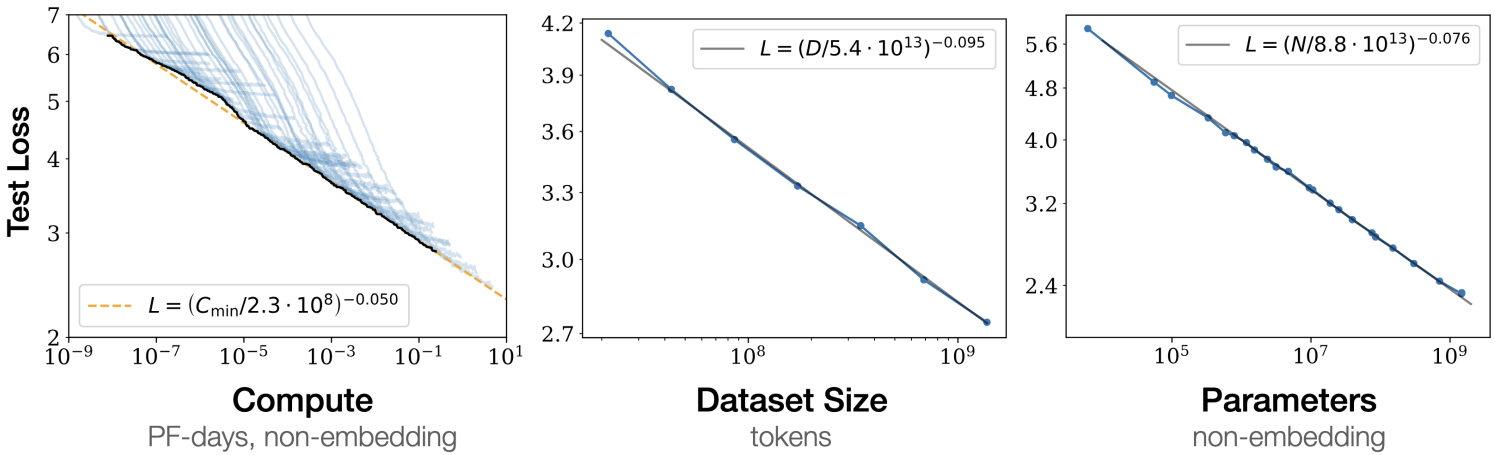
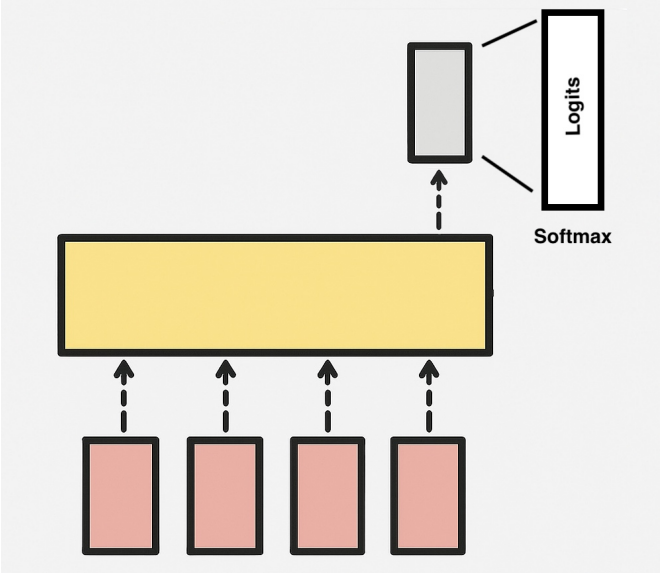


Figure 1 from Kaplan et al. (2020)

Outline

- Foundations
- Sequence models and recurrent neural networks
- The transformer architecture
- Large language (base) models
  - Overview
  - Pre-training
  - Inference

Inference: Generating new texts



- The core of an LLM consists of code to compute the forward pass and a large number of floating point numbers (parameters)
- Recall our example sentence, "Language models are interesting", which was tokenised into $T = 4$ tokens with *tiktoken o200k_base*
- After being processed through all layers of the transformer (yellow), the last token's embedding is mapped into a vector with as many elements as there are tokens in the vocabulary, and a softmax is applied
- A token is sampled given these probabilities (potentially scaled by a *temperature* parameter) and appended to the context

Inference: Generating new texts

- Step 1:
  - **Input (initial prompt):** $[17238, 7015, 553, 9559]$ - "Language models are interesting"
  - **Sampled token:** $[53081]$ - "because"
- Step 2:
  - **Input:** $[17238, 7015, 553, 9559, 53081]$ - "Language models are interesting because"
  - **Sampled token:** $[33574]$ - "they"
- Step 3:
  - **Input:** $[17238, 7015, 553, 9559, 53081, 33574]$ - "Language models are interesting because they"
  - **Sampled token:** $[553]$ - "are"
- ...

Examples

- Some APIs allow querying base models, but these models auto-complete and generate internet texts by construction
- Large models have stored so much information in their parameters that they can recite some passages verbatim
- *In-context learning* can already allow base models to respond as assistants in some stylised cases by implying the answer as the natural text completion of the prompt

```
You are a helpful AI assistant answering questions.
User question: What is the abbreviation of the
London School of Economics and Political Science?
Your answer: LSE
User question: When was the LSE founded?
Your answer: 1895
User question: Why is the sky blue?
Your answer:
```

Discussion and outlook

- So far we have obtained a kind of internet document simulator
- In addition, while the test loss of base training is correlated with downstream capabilities, compute keeps increasing (Moore's law) but internet data is running out
- Beyond focusing on higher quality training data or adding synthetic data, this increasingly emphasises the importance of post-training
- Post-training trains the models with approaches such as supervised fine-tuning and reinforcement learning to respond to questions, and aligns them more broadly as well as enhances their capabilities
- We will discuss these topics in the next lecture

Minimal GPT without PyTorch

- For a recent minimal implementation of a GPT model from scratch in ∼200 lines of Python code without dependencies, see `https://karpathy.github.io/2026/02/12/microgpt/` and `https://gist.github.com/karpathy/8627fe009c40f57531cb18360106ce95`
- It is a great resource for further study of the architecture, base training, and inference with a minimal example

Resources for further study and references for these slides

- Detailed applied lecture on large language models: https://www.youtube.com/watch?v=7xTGNNLPyMI
- Visualisation of the GPT model and attention mechanism: https://www.youtube.com/watch?v=wjZofJX0v4M
- Written introductions to transformers: https://jalammar.github.io/illustrated-transformer/ or https://peterbloem.nl/blog/transformers
- Lecture series on building neural networks and language models from scratch: https://www.youtube.com/watch?v=VMj-3S1tku0&list=PLAqhIrjkxbuWI23v9cThsA9GvCAUhRvKZ

References I

Bahdanau, Dzmitry, Kyunghyun Cho, and Yoshua Bengio, "Neural machine translation by jointly learning to align and translate," *arXiv preprint arXiv:1409.0473*, 2014.

Bengio, Yoshua, Réjean Ducharme, Pascal Vincent, and Christian Jauvin, "A neural probabilistic language model," *Journal of machine learning research*, 2003, *3* (Feb), 1137–1155.

Choromanski, Krzysztof, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser et al., "Rethinking attention with performers," *arXiv preprint arXiv:2009.14794*, 2020.

Dao, Tri, "Flashattention-2: Faster attention with better parallelism and work partitioning," *arXiv preprint arXiv:2307.08691*, 2023.

—, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré, "Flashattention: Fast and memory-efficient exact attention with io-awareness," *Advances in neural information processing systems*, 2022, *35*, 16344–16359.

References II

Devlin, Jacob, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.

Kaplan, Jared, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei, "Scaling laws for neural language models," *arXiv preprint arXiv:2001.08361*, 2020.

Karpathy, Andrej, "Lecture: Intro to Large Language Models," 2023. `https://www.youtube.com/watch?v=zjkBMFhNj_g`.

__ , "Stanford CS25: Introduction to Transformers w/ Andrej Karpathy," 2023. `https://www.youtube.com/watch?v=XfpMkf4rD6E`.

__ , "Lecture: Deep Dive into LLMs like ChatGPT," 2025. `https://www.youtube.com/watch?v=7xTGNNLPyMI`.

References III

Liu, Yinhan, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov, "Roberta: A robustly optimized bert pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019.

Mikolov, Tomas, Kai Chen, Greg Corrado, and Jeffrey Dean, "Efficient estimation of word representations in vector space," *arXiv preprint arXiv:1301.3781*, 2013.

Pennington, Jeffrey, Richard Socher, and Christopher D Manning, "Glove: Global vectors for word representation," in "Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)" 2014, pp. 1532–1543.

Radford, Alec, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever et al., "Language models are unsupervised multitask learners," *OpenAI blog*, 2019, *1* (8), 9.

References IV

Shah, Jay, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao, "Flashattention-3: Fast and accurate attention with asynchrony and low-precision," *Advances in Neural Information Processing Systems*, 2024, *37*, 68658–68685.

Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, 2017, *30*.

Appendix

Dot product attention: Details on queries and keys

Back

- To compute e.g. $\alpha_{1,2}$:

  1. **Query vector:** Compute the query for token 1

  $$x_1^Q = W^Q x_1$$

  2. **Key vectors:** For every token $t = 1, \ldots, T$, compute

  $$x_t^K = W^K x_t$$

  3. **Attention weight:** Calculate

  $$\alpha_{1,2} = \frac{\exp\left(\frac{x_1^Q \cdot x_2^K}{\sqrt{d}}\right)}{\sum_{t=1}^{T} \exp\left(\frac{x_1^Q \cdot x_t^K}{\sqrt{d}}\right)}$$

  where d is the dimension of the key and query vectors

- All other weights $\alpha_{t,n}$ are computed analogously
- The attention block is characterised by W^V, W^Q, W^K
- There will be an exercise in the next class!

Dot product attention: Intuition and challenges

- Intuition
  - Query: What information is a token seeking
  - Key: What information is a token offering
  - Dot product is a measure of similarity
- Challenges
  - Expensive in terms of computation and memory due to quadratic complexity - need to compute $T \times T$ dot products
  - This makes it difficult to scale to very long context windows
  - Possible variants: Improve memory access (e.g. Dao et al., 2022; Dao, 2023; Shah et al., 2024), decrease quadratic complexity (e.g. Choromanski et al., 2020)